



Reliability in Event-Driven Design

How Correctness Boundaries Shape Process-Handling Choices and Recovery Behaviour

Herman Karlsson

This thesis is submitted to the Faculty of Computer Science at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Higher Education Diploma in Software Engineering. The thesis is equivalent to 10 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author:

Herman Karlsson

E-mail: hekr23@student.bth.se

University advisor:

Ramin Irani

Department of Computer Science

Faculty of Computer Science
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Reliability in event-driven processes is difficult to judge when the conditions that make a process acceptable are left implicit. An event-driven process may preserve work, delay it, replay it, duplicate it, or omit it, but these behaviours do not have a fixed process-level meaning. A retained event may arrive too late to help. A missed update may no longer matter. Duplicate control may reduce immediate harm while shifting work into later settlement. The value of a handling choice therefore depends on the condition it is meant to satisfy.

Objectives. This thesis develops a boundary-first framework for reasoning about reliability in event-driven processes. The aim is to make explicit what a process must protect before judging which process-handling model is appropriate.

Methods. The thesis follows a design science research approach. It develops the framework and examines it through a controlled empirical suite that compares three correctness-boundary classes with three process-handling models. The boundary classes are deadline-constrained, state-oriented, and required-effect. The handling models are transient/immediate, retained/immediate, and retained/deferred. The suite narrows the setting so that selected differences in preservation, replay, repayment, and settlement can be observed under defined disturbances.

Results. Across the controlled scenarios, the comparison shows that the value of replay, retention, immediate handling, and deferred settlement depends on the correctness boundary at stake. For deadline-constrained work, recovery matters only while unsettled work can still be repaid before expiry. For state-oriented work, replay matters when missed state remains relevant, while later state may overtake earlier misses. For required-effect work, replay can recover downstream handling gaps after emission, but not source omission. Deferred settlement can reduce immediate duplicate pressure, but it does so by moving part of the burden into later reconciliation.

Conclusions. The thesis contributes a process-level framework for comparing handling choices under explicit correctness requirements. Its value lies in making the comparison inspectable: what condition or set of conditions is at stake, what can disrupt or complicate them, which handling model is chosen, and what trade-off follows. The guide gives this reasoning a more concrete form, so that the design judgments behind handling choices can be understood more clearly.

Keywords: event-driven processes, reliability, design science research, correctness boundaries, process-handling models

Contents

Abstract	i
1 Introduction	1
1.1 Background and motivation	1
1.2 Problem, scope, and conceptual starting point	1
1.3 Research questions	2
1.4 Outline of the thesis	2
2 Related Work	4
2.1 Event-driven work as case-level obligation	4
2.2 Mechanisms and process-level correctness	5
2.3 Three working boundary classes in context	6
2.4 Disturbance as an analytical lens	7
2.5 Positioning and contribution	7
3 Method	9
3.1 Research design	9
3.2 Experimental environment	9
3.3 Scenario and run structure	10
3.4 Operationalising the correctness boundaries	11
3.5 Metrics and interpretation	12
3.6 Reporting discipline and scope	13
4 Results and Analysis	14
4.1 Deadline-constrained results	14
4.2 State-oriented results	17
4.3 Required-effect results	18
5 Discussion	21
5.1 Answer to RQ1: which conditions define correctness across different event-driven processes?	21
5.2 Answer to RQ2: how do process-handling models shape preservation, replay, repayment, and settlement under disturbance?	22
5.3 Answer to RQ3: correctness boundaries and process-handling choice .	23
5.3.1 Boundary-first design guide	23
5.3.2 Producer omission and producer-integrity controls	24
5.3.3 Illustrative design examples	24
5.3.4 Scope and realism	26

5.4	Relation to established work	26
5.5	Limitations and claim boundaries	27
5.6	Ethical and professional considerations	28
6	Conclusions and Future Work	29
6.1	Conclusion	29
6.2	Placing the thesis in perspective	29
6.3	Future Work	30
A	Boundary-first design guide	32
B	Supplementary design examples	33
C	Empirical suite synopsis	34
	Supplementary material	41
	References	42

List of Figures

4.1	Deadline outcome burden	15
4.2	Runtime resolution and final deadline burden	16
4.3	Cumulative backlog-shock accounting	17
4.4	State-oriented adequacy	18
4.5	Source omission boundary by received events	19
4.6	Duplicate-pressure containment	19
C.1	Empirical suite execution pipeline	35

List of Tables

3.1	Frozen empirical suite manifest.	11
3.2	Operationalisation of correctness boundaries.	12
3.3	Boundary-aligned metric groups.	13
4.1	Within-suite duplicate-containment trade-off under deferred settle- ment, shown as rounded means across ten repetitions.	20
5.1	Compact use of the boundary-first guide.	23
A.1	Boundary-first guide for process-handling choice.	32
B.1	Design scenario summary	33
C.1	Operational definitions of reported metrics.	37
C.2	Scenario and parameter specification.	38
C.3	Disturbance-to-channel map.	39
C.4	Evaluation controls and scope protection.	40

1.1 Background and motivation

Event-driven design often carries work across time and across component boundaries. An order, update, payment, booking, notification, or intervention may begin in one part of a system and be completed somewhere else, later, through an event flow. This provides useful flexibility: components can be decoupled, uneven demand can be absorbed, and work can continue asynchronously. It can also make reliability harder to judge, because the outcome of a process is no longer visible from event movement alone.

A mechanism may work as intended and still leave the relevant process outcome uncertain. A retained event may arrive after the useful window has closed. A replayed update may have been overtaken by later information. Duplicate control may prevent repeated side effects while still leaving reconciliation or delayed settlement behind. In such cases, the mechanism may have done something useful, but it still has to be read against the process result it is meant to help secure.

The concern is not only conceptual. It is also visible in the way event-driven systems are studied and built. Work on event-driven and microservice systems reports practical difficulty with schemas, auditing, ordering, consistency, and transactional messaging support [1, 2]. Stream-processing research likewise treats event processing as a setting where correctness, latency, and cost have to be considered together [3].

Some complexity is unavoidable. Event-driven processes separate initiating causes from later effects, and make correctness depend on timing, state, delivery, replay, duplication, and settlement. One useful response is to take a step back, consider one process at a time, and begin with a deceptively simple question: what would make this process count as acceptable?

The thesis takes this question as its starting point. It gives a name and structure to this prior judgment, so that handling choices can be compared against the condition or set of conditions the process has to satisfy.

1.2 Problem, scope, and conceptual starting point

The name used here for such a condition is a *correctness boundary*. A correctness boundary is a condition that must hold for a case to count as acceptable. A process may have one such boundary or several. The point is to reason about those conditions clearly, both separately and together, before the handling choice is judged.

The term keeps the condition at stake distinct from the technical means used to meet it. Those means include event-transport capabilities such as publication, delivery, retention, and replay, and application-level safeguards such as idempotence and settlement logic. A process-handling model is the arrangement of these means for a given process. The design question is which handling model is appropriate to the condition or set of conditions that process must satisfy.

To make the comparison concrete, the analysis focuses on three working boundary classes. In deadline-constrained work, correctness depends on reaching a valid outcome before a window closes. In state-oriented work, correctness depends on whether missed or delayed updates still matter to the state the process is required to preserve or expose. In required-effect work, correctness depends on whether one downstream action happens reliably and without harmful duplication. These classes name recurring correctness conditions rather than mutually exclusive types of real-world process. Their value is analytical: they help show what a handling choice is being asked to protect, including where a process combines more than one condition.

The empirical comparison uses three process-handling models. In transient/immediate handling, work depends on being handled as it arrives. In retained/immediate handling, emitted work remains available for replay after a handling gap. In retained/deferred handling, emitted work is preserved first and settled later. These models do not exhaust event-driven handling, but they make preservation, replay, repayment, and settlement visible under controlled disturbance.

The thesis is therefore design-oriented, but restrained in its claim. It develops a boundary-first framework for judging process-handling models against explicit correctness boundaries and examines that framework through a controlled empirical suite. The suite narrows the setting so that selected handling differences can be observed clearly under defined disturbances. Its role is to support the comparison, not to reproduce the full complexity of deployed event-driven systems or mirror their full range of operating conditions.

1.3 Research questions

The thesis is guided by three research questions.

RQ1. Which conditions define correctness across different event-driven processes?

RQ2. How do process-handling models shape preservation, replay, repayment, and settlement under disturbance?

RQ3. How can explicit correctness boundaries guide process-handling choices in event-driven design?

1.4 Outline of the thesis

Chapter 2 situates the thesis in relation to the literature most relevant to event-driven reliability, correctness, recovery, and coordination. Chapter 3 explains the research design, the empirical suite, the interpretive metrics, and the study's scope limits. Chapter 4 presents the empirical results across the three working boundary classes. Chapter 5 discusses what those results mean in relation to the research questions,

the design guidance developed from them, the established literature, and the thesis's claim boundaries. Chapter 6 closes the thesis and points to future work.

This chapter positions the argument within the bodies of work most directly relevant to it. Chapter 1 introduced the thesis’s boundary-first vocabulary and scoped the study. The task here is to relate that vocabulary to the strands of distributed-systems and event-driven literature that matter most for how correctness, recovery, and process-handling choices are understood.

The study sits between established distributed-systems research and practical event-driven design. Prior work has given rich accounts of publish-subscribe decoupling, end-to-end reasoning, fault recovery, tail behavior, replication, stream processing, and compensating action [3, 5, 6, 7, 10, 11, 12, 13]. These works provide the background for the argument developed here. They show that reliability is never only a matter of moving messages. It is also a matter of what a system is trying to preserve, how that preservation is attempted, and what costs follow from the chosen form of handling.

These bodies of work make the starting point concrete. Event-driven design already offers many ways to publish, retain, replay, process, coordinate, and settle work. What remains difficult is how those means should be judged when a designer must decide what kind of correctness is actually at stake in a concrete flow. This matters all the more as event-streaming and stream-processing infrastructure have become more common and prominent in both research and practice [13]. The more readily such support is available, the easier it becomes to reason from available capability while leaving the foundational question unstated: what, in concrete terms, must still hold for the process to count as correct?

In practical system discussions, the same vocabulary is often used for quite different obligations. Replay, retention, immediate handling, idempotence, eventual consistency, and compensation all matter, but they do not matter in the same way for every flow. Work on event-management challenges and dataflow trade-offs suggests that correctness concerns are often more varied than the language around them admits [1, 3, 14]. This thesis starts from that variation.

2.1 Event-driven work as case-level obligation

The unit of analysis here is not the event in isolation. It is the case or process whose acceptability may depend on what the event causes, preserves, updates, or triggers. An event is important because it participates in some larger piece of work. It may report a change in state, trigger a downstream action, carry work forward, or make

recovery possible after a handling gap. The same technical treatment of an event can therefore be adequate in one flow and inadequate in another.

This is why event delivery alone is too narrow a view of reliability. A delivered event may arrive too late to help. A retained event may be replayable after the point at which its result can still count as correct. A missed state update may be harmless if later state legitimately overtakes it. A downstream action may still be wrong if it happens twice, even when every individual message was processed according to the mechanism's local rules. In each case, the relevant question is not only whether the event moved through the event flow. The question is whether the process still satisfies the condition that makes it acceptable.

This case-level view extends an end-to-end style of reasoning into event-driven process design. Saltzer, Reed, and Clark show that a lower-level mechanism may be useful and still be insufficient if the final requirement must be checked at the application boundary [5]. The same discipline applies here. Replay, retention, idempotence, and deferred settlement are not strong or weak in themselves. They are adequate or inadequate in relation to what the process must protect. The contribution is to express that discipline in process-facing terms for event-driven reliability.

2.2 Mechanisms and process-level correctness

The distinction introduced in Chapter 1 matters because reliability mechanisms do not carry a fixed process-level meaning on their own. Replay, for example, means that emitted events can be processed again. That does not by itself say whether replay protects the process. Replay can recover emitted work after a downstream handling gap, but it cannot recover an event that was never emitted. It can preserve work for later completion, but that may not help if the work settles after a deadline has passed. Its value depends on the condition it is used to protect and on the handling model in which it operates.

The same is true of retention, immediate handling, deferred settlement, idempotence, producer-side emission safeguards, and reconciliation. Each matters only in relation to the condition it is used to protect. A retained stream matters when preserved work remains relevant. Immediate handling matters when delay or loss would invalidate the case. Deferred settlement matters when later reconciliation is acceptable and when reducing harmful duplication is worth the cost it introduces. Producer-side safeguards matter when the event must enter the flow before any downstream mechanism can act on it.

A process-handling model is therefore not a single mechanism. It is the arrangement through which the process attempts to keep the relevant condition or set of conditions true. In the empirical part of the thesis, this idea is represented by three basic handling models: transient/immediate, retained/immediate, and retained/deferred. They do not exhaust the possible forms of event handling, but they make the central design contrast visible. They show that a handling choice has to be judged against what the process must protect.

2.3 Three working boundary classes in context

The three boundary classes used in the thesis are working cases rather than a complete taxonomy. They were chosen because they make common event-driven correctness conditions visible: timely completion, acceptable state, and one downstream action happening reliably and without harmful duplication. Each class connects to concerns already present in distributed-systems and event-driven literature, including timeliness, state consistency, recovery, idempotence, compensation, and transactional event publication [3, 6, 7, 14, 23]. The thesis brings these concerns together as a process-level way of judging handling choices.

The classes are analytical distinctions. They describe the condition being examined, not a classification of the whole flow. A real process may carry more than one protected condition. For example, a payment authorisation may need to complete while the checkout remains valid and also avoid harmful repeated execution. The design task is therefore to state what must remain true for the process to count as acceptable, and then judge whether the handling choice protects that condition or set of conditions.

In deadline-constrained work, the protected condition is timely completion. A case must reach a valid outcome while that outcome still counts. This makes delay and repayment central. Preserved work is not enough if it settles after the validity window has closed. For this kind of work, the decisive question is whether recovery can still arrive in time.

In state-oriented work, the protected condition is an acceptable visible or resulting state. The relevant question is not whether every intermediate update was processed, but whether missed or delayed updates still matter to the state the process is required to preserve or expose. Later updates may overtake missed information and restore the condition that matters. In other cases, a missed update may leave stale or misleading state in force during the disturbance, even if the final state later becomes acceptable.

In required-effect work, the protected condition is whether one downstream action happens reliably and without harmful duplication. This boundary makes it necessary to separate producer integrity, downstream recovery, duplicate control, and settlement cost. If a valid event has been emitted, replay can recover a downstream handling gap. If the event was never emitted, downstream replay has no object to act on. Producer-side safeguards are therefore important, but they protect entry into the event flow rather than forming a fourth process-handling model. If repeated triggers or redelivery can cause repeated effects, idempotence or deferred settlement may be needed, but those mechanisms address a different risk and may shift cost into reconciliation or delayed settlement.

These classes matter because they keep reliability tied to the condition at stake rather than to the event substrate in general. The same handling choice can have different significance depending on whether the process must finish in time, preserve an acceptable state, or carry a downstream action through without loss or harmful duplication.

2.4 Disturbance as an analytical lens

The empirical work focuses on disturbance because smooth event flow often hides the distinctions the framework is meant to reveal. When events are emitted, delivered, handled, and settled without delay or loss, different handling models can appear equivalent. The process succeeds, and the reason for success is not very informative. The harder question is what happens when delay, loss, duplicate pressure, omission, or backlog interrupts that ordinary path.

Disturbance is therefore not an exceptional corner of the argument. It is the lens through which the consequences of a handling model become visible. Under disturbance, the comparison reveals whether work was merely delivered or actually protected. It shows whether retained work can still settle in time, whether missed state still matters, whether a downstream handling gap remains replayable, and whether duplicate containment shifts cost into later reconciliation.

Under disturbance, the same mechanism can take on different significance. In a smooth run, replay may never be called upon, so its practical value remains latent. Under a downstream handling gap, retained replay can recover work that transient handling loses. Under source omission, the same replay capability has nothing to act on. Under a deadline boundary, replay may preserve work that still arrives too late to count. Under a state boundary, replay may be unnecessary if later state has already restored the condition that matters. Replay is therefore not simply good or bad. Its value depends on the boundary being protected and on the handling model in which it operates.

This also explains the attention given to repayment and settlement, not only to final success. A case can eventually complete while still exposing an unacceptable delay. A process can end in an adequate state while still exposing incomplete or misleading state during the outage. A deferred step can restrain harmful duplicates while adding reconciliation work and delayed settlement. Disturbance makes these differences observable. It turns mechanism choice into a question of what was protected, what was merely postponed, and what cost was moved elsewhere.

2.5 Positioning and contribution

The contribution of the thesis is a boundary-first way of reasoning about reliability in event-driven processes. It starts from a simple distinction: before a handling model can be judged, the condition or set of conditions the process must protect has to be made explicit.

This places the thesis close to end-to-end reasoning. Saltzer, Reed, and Clark show that a mechanism may be useful at one level and still be insufficient unless the final requirement is protected at the level where it can actually be checked [5]. The same discipline applies here. Replay, retention, idempotence, deferred settlement, and producer-side safeguards are interpreted through the condition they are meant to protect in the specific process being judged, rather than as general answers to reliability.

The argument is also close in spirit to work on coordination avoidance and CALM. That work relates coordination to the property being protected, such as application-

level invariants or the monotonicity of a computation [8, 9]. It makes a similar move at the level of event-driven processes. It asks what a given process must protect before judging how much preservation, replay, coordination, or settlement is justified.

Stream-processing and dataflow work provide a further point of contact. They show that correctness, latency, and cost interact, and that the value of an output depends on timing, correction, and later updates [3, 13]. Here, that trade-off is expressed at the level of a particular event-driven process. Where a deadline-constrained condition is at stake, preserved work matters only while it can still settle in time. Where a state-oriented condition is at stake, replay matters when missed state remains relevant. Where a required-effect condition is at stake, downstream recovery, duplicate control, producer integrity, and settlement cost answer different parts of the problem.

The practical setting for the thesis comes from event-driven and microservice systems, where designers already face questions about schemas, event-flow auditing, consistency, recovery, transactional messaging, idempotence, sagas, compensation, and event publication [1, 2, 7, 14, 15, 16, 23]. These areas provide many useful mechanisms and patterns. The contribution here is to make their relevance more explicit for a particular process: what condition or set of conditions is this process protecting, how could those conditions fail, and which handling choice is proportionate?

The thesis should therefore be read as a complementary clarification within established work on correctness, recovery, stream processing, and coordination. Existing work gives many accounts of mechanisms, guarantees, and patterns. The contribution here is to connect those concerns at the level of a particular event-driven process: what condition or set of conditions must hold, how those conditions could fail, and which handling choice is proportionate.

3.1 Research design

The thesis follows a design science research approach: it develops a boundary-first framework and examines it through a controlled empirical suite [17, 18]. In design-science terms, the framework is the central artifact, while the guide developed later gives it a more concrete form. The guide is not treated as a separately validated design method. The empirical role of the method is narrower: it makes selected consequences of process-handling choices observable under defined correctness boundaries and disturbances.

The empirical suite abstracts from full system complexity so that preservation, replay, repayment, and settlement can be compared under controlled disturbance. It follows the structure developed in Chapter 2: correctness boundaries define what is at stake, process-handling models define how work is handled, and disturbance makes the consequences of those choices observable.

3.2 Experimental environment

The empirical suite is implemented as a small Docker-based event-processing environment. Docker is used to keep the runtime arrangement repeatable and to separate the main roles in the experiment: event production, event transport, consumer handling, retained replay, deferred settlement, and reporting [19]. The environment is compact by design, because the study depends on comparing selected handling behaviours clearly.

Redis provides the event substrate. Redis Pub/Sub is used for transient publication, where events are delivered to active subscribers but are not retained for later replay [20]. Redis Streams is used for retained event handling, where emitted events remain available after initial delivery and can therefore support replay [21]. This combination gives the suite the contrast needed for the comparison: handling that depends on arrival-time processing, handling that can recover emitted work through replay, and handling that separates preservation from later settlement.

The consumers implement the handling behaviour examined in the suite. In immediate handling, work is attempted as events arrive or are replayed. In deferred handling, emitted work is preserved first and resolved through a later settlement step. Reporting scripts collect run outputs and derive the endpoint, runtime, replay, state, and settlement measures used in the analysis.

Disturbance is treated in the same abstracting spirit. The suite does not model every concrete reason why handling might slow down or fail. It represents the effects that matter for the comparison: processing takes longer, work accumulates, emitted work is missed, repeated triggers bear on the same downstream obligation, or some intended cases are never emitted as events. Source omission is kept separate because replay has nothing to recover when the event never enters the flow.

3.3 Scenario and run structure

The suite is organised around the three working boundary classes developed in Chapter 2. Each module keeps the scenario structure small enough to interpret, while varying the disturbance most relevant to that boundary.

The deadline-constrained module examines timely completion. It uses four scenario families: baseline, moderate degradation, high degradation, and backlog shock. Baseline provides the reference condition. Moderate and high degradation represent sustained consumer-side pressure at two intensities. Backlog shock represents a sharper disturbance in which work accumulates and must later be repaid. These scenarios are compared across the three process-handling models: transient/immediate, retained/immediate, and retained/deferred.

The state-oriented module examines whether missed or delayed updates still matter to the resulting or visible state. The comparison is smaller than in the deadline module because the empirical question is narrower: whether replay contributes to final or exposed state adequacy when updates are missed or delayed.

The required-effect module examines whether one downstream action happens reliably and without harmful duplication. It uses handling gap, source omission, and duplicate pressure to separate three concerns: replayable downstream loss, producer-side non-emission, and repeated triggers bearing on the same downstream obligation.

Each scenario cell combines a boundary module, a disturbance profile, and a process-handling model. A single run then executes that configured cell from start to finish: the producer emits the workload, the disturbance affects handling according to the profile, the selected model processes or preserves the work, and the reporting scripts record both endpoint outcomes and runtime traces. This structure keeps the unit of comparison stable across the suite. The comparison is not between isolated events, but between repeated executions of the same scoped process condition under the same disturbance and handling model. The full scenario and parameter specification is reported in Appendix C, Table C.2.

Each reported scenario cell is repeated ten times. The repetitions are used to avoid treating a single execution trace as decisive and to check that the comparative pattern remains visible across runs. Endpoint figures therefore use aggregate summaries where appropriate, while runtime figures show median traces when the shape of recovery or repayment over time is the point of interpretation. Appendix C gives a short account of suite run stability across repeated full runs.

Table 3.1: Frozen empirical suite manifest.

Module	Scenario cells	Repetitions per cell	Completed runs	Role in the argument
Deadline-constrained	12	10	120	Examines repayment, expiry burden, and runtime-versus-endpoint contrast.
Required-effect	18	10	180	Examines replayable handling gaps, source omission, duplicate containment, and settlement cost.
State-oriented	4	10	40	Examines latest-state adequacy, outage-time exposure, and forward resumption.
Total	34	10	340	Frozen reporting suite used for the thesis comparison.

The disturbance profiles serve different comparative purposes. Moderate degradation introduces sustained processing pressure, high degradation intensifies that pressure, and backlog shock concentrates delay into an interruption after which accumulated work may still be repaid before expiry.

3.4 Operationalising the correctness boundaries

Table 3.2 shows how the three boundary classes are made observable in the suite. Each module isolates one defining empirical question, so that the handling models can be compared against the condition at stake.

Table 3.2: Operationalisation of correctness boundaries.

Boundary class	Empirical question	Main observations
Deadline-constrained	Can affected cases still reach a valid outcome before the window closes?	Attained and expired outcomes, unresolved and doomed burden, peak and integrated burden, repayability, and run-time traces.
State-oriented	Does the resulting or visible state remain acceptable despite missed or delayed updates?	Final state adequacy, outage-time exposure, and later state overtaking missed updates.
Required-effect	Does one downstream action happen reliably and without harmful duplication?	Unattained effects, handling-gap recovery, source omission, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay.

The measures are therefore boundary-aligned rather than generic. Deadline-constrained scenarios pair endpoint outcomes with runtime burden. State-oriented scenarios separate final adequacy from outage-time exposure. Required-effect scenarios separate downstream recovery, source omission, duplicate containment, and settlement cost.

3.5 Metrics and interpretation

The metrics are used to read endpoint outcomes together with the path by which those outcomes were reached. Endpoint measures show whether the relevant condition was met. Runtime and settlement measures show how that result came about: whether work accumulated, whether recovery arrived in time, whether state was visible during disturbance, and whether duplicate containment shifted cost into later reconciliation. Table 3.3 summarises the metric groups used in the analysis. The full operational definitions are provided in Appendix C, Table C.1.

Table 3.3: Boundary-aligned metric groups.

Boundary class	Main empirical reading	Supporting measures
Deadline-constrained	Whether cases can still reach a valid outcome before the deadline closes.	Deadline miss rate, completion-in-time rate, unresolved burden, doomed burden, repayment pattern, and resolution pulse.
State-oriented	Whether final state adequacy and outage-time exposure tell the same story.	Latest-state attainment rate and outage-exposed seen fraction.
Required-effect	Whether the downstream action happens reliably and without harmful duplication.	Required-effect attainment, received-event count, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay.
Cross-cutting interpretation	Whether the final outcome alone hides the path by which it was reached.	Endpoint outcomes read together with runtime traces, replay signals, and settlement-cost measures.

The main interpretive point is that final outcomes are not enough on their own. Completion, convergence, or duplicate containment may look favourable while still arriving too late, exposing misleading state during a disturbance, or shifting cost into later settlement. For that reason, the results are read through endpoint outcomes, runtime traces, and settlement measures together. This emphasis on runtime meaning is consistent with recovery-oriented thinking and with recent benchmarking work on fault recovery behaviour [11, 12, 15, 16].

3.6 Reporting discipline and scope

The reported results come from the frozen suite described above: fixed scenario definitions, fixed output extraction, fixed plotting conventions, and fixed reporting measures. Preliminary runs were used to check that the main comparative patterns remained stable enough to support the argument. The thesis therefore treats the empirical material as a controlled comparison across defined conditions.

Within this abstraction, the suite supports comparative interpretation. The measured values are read as evidence about the relative behaviour of the handling models under the defined disturbances, not as transferable performance values for external systems or workloads. The narrower claim is that the value of replay, retention, immediate handling, and deferred settlement depends on the correctness boundary being protected.

The results are organised by correctness boundary. Each section asks how the handling models behave when the relevant condition is deadline-constrained, state-oriented, or required-effect. Across the chapter, outcomes are interpreted through the condition at stake, including whether recovery still arrived in time, whether recovered state still mattered, and whether duplicate containment left an acceptable settlement burden.

4.1 Deadline-constrained results

The central question in this section is whether cases affected by disturbance can still be brought to a valid outcome before the deadline passes. From that standpoint, backlog becomes analytically important when it threatens timely completion. The key issue is not simply whether work remains available, but whether unsettled work can still be repaid before the validity window closes [3, 12].

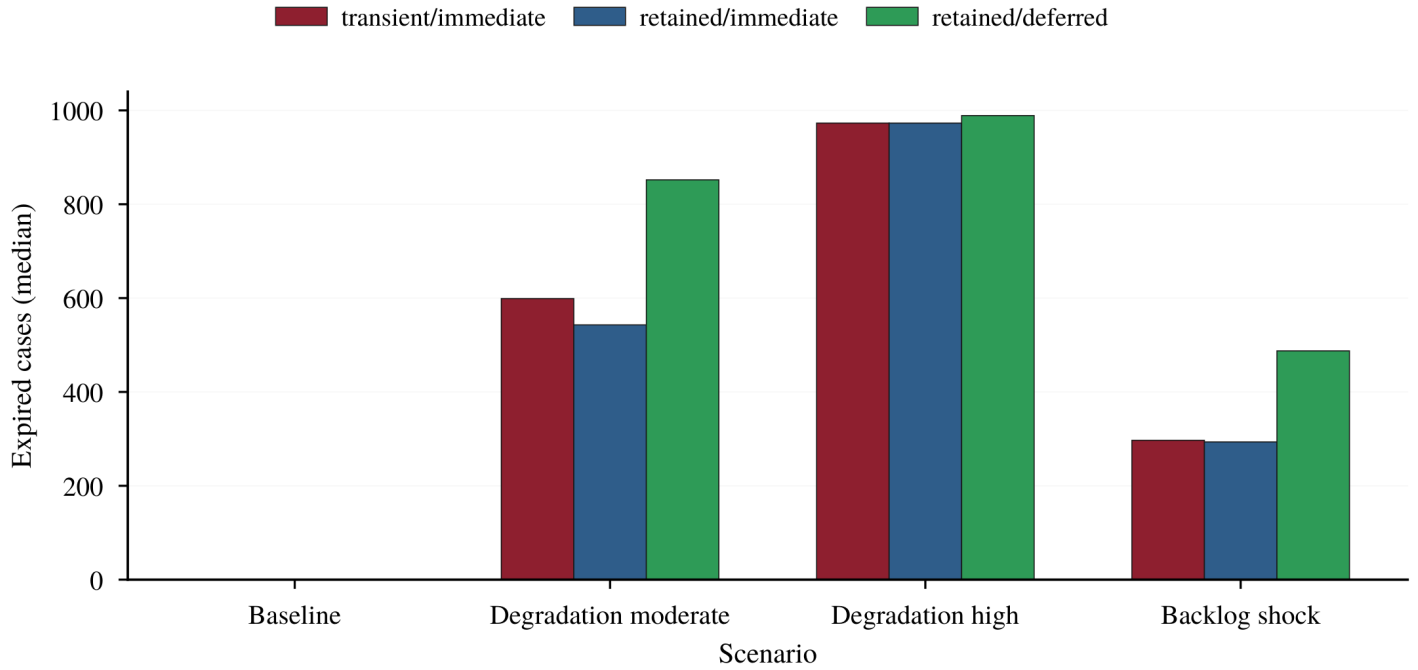


Figure 4.1: Deadline outcome burden across the canonical scenario spine. The bars show the median number of expired cases across ten repetitions for baseline, backlog shock, moderate degradation, and high degradation under the three process-handling models.

The endpoint view shows that deadline harm is shaped by repayability, not by disturbance intensity alone. Baseline produces no deadline burden, while backlog shock and sustained degradation produce different levels of expiry. A short, sharp disruption can be less damaging than a sustained slowdown if enough time remains for repayment. For deadline-constrained work, preserved work matters only if it can still settle before the deadline closes.

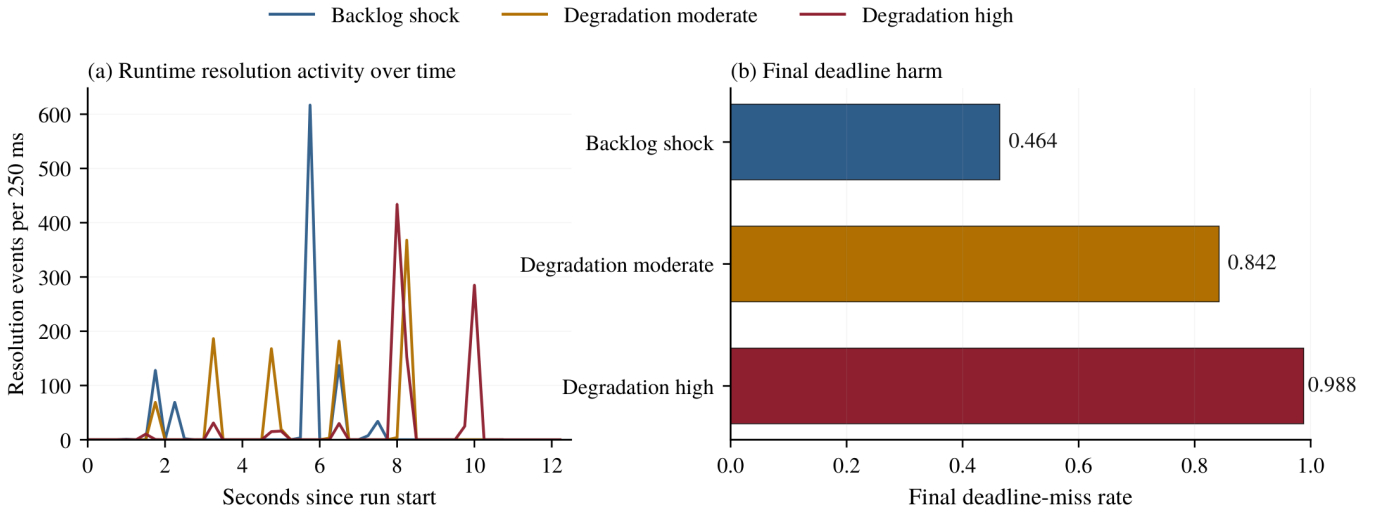


Figure 4.2: Runtime resolution activity and final deadline burden under retained/deferred handling. Panel (a) shows completions and expiries per 250 ms bin as median traces across the ten repetitions in the frozen main-core batch, while panel (b) shows the corresponding median final deadline-miss rate.

The runtime view explains why. Under retained/deferred handling, backlog shock produces the most visible burst of resolution activity, but not the greatest final deadline harm. Moderate and high degradation are less visually dramatic, yet they leave more cases beyond the deadline because they reduce the time available for repayment throughout the run. The figure therefore shows why runtime activity and endpoint damage have to be interpreted together.

Within the configured disturbance profiles and expiry window, the value of the contrast is illustrative: it shows how peak runtime pressure and final deadline harm can diverge when accumulated work can still be repaid before expiry.

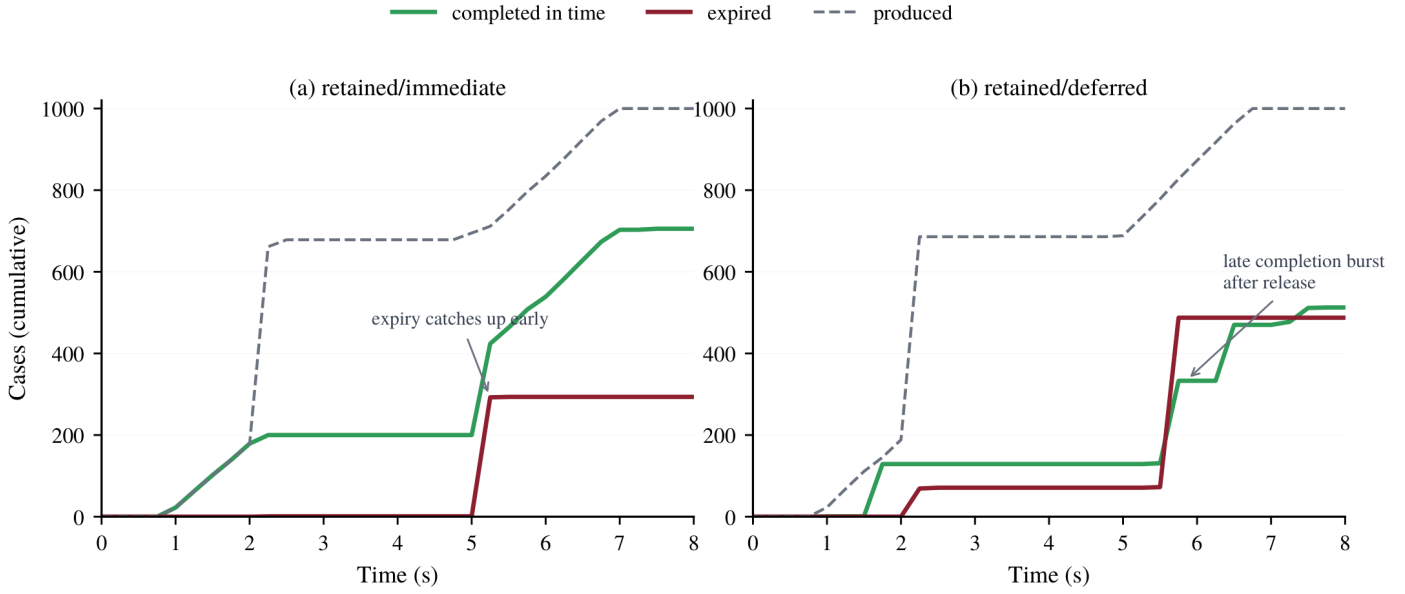


Figure 4.3: Cumulative accounting for the backlog-shock case. The curves show produced cases, in-time completions, and expiries as median cumulative traces across the ten repetitions in the frozen main-core batch.

The cumulative view gives a more concrete accounting of the backlog-shock case. The retained/deferred trace includes a visible late completion burst after release, but much of the deadline damage has already occurred by then. Recovery activity is therefore not enough on its own. For this boundary, it matters only if it arrives while the outcome can still count as correct.

Taken together, the figures show that deadline-constrained work is governed by repayability before expiry, not by preserved work or peak runtime pressure alone. Retention and replay can keep work available, but the handling model should be judged by whether recovery can still arrive in time for correctness to hold.

4.2 State-oriented results

State-oriented work raises a different correctness question: whether the state represented by the process remains acceptable despite missed or delayed updates. From that standpoint, replay is not valuable in itself. Its value depends on whether a missed update remains relevant to the state the process is required to preserve [6, 22].

Two measures separate final adequacy from outage-time exposure. Latest-state attainment records whether the process reaches the expected latest version for the case. Outage-exposed seen fraction records whether updates during the disturbance were observed. The distinction matters because a process may end in an acceptable state even when some intermediate state was missed, while in other cases the missed state may remain relevant.

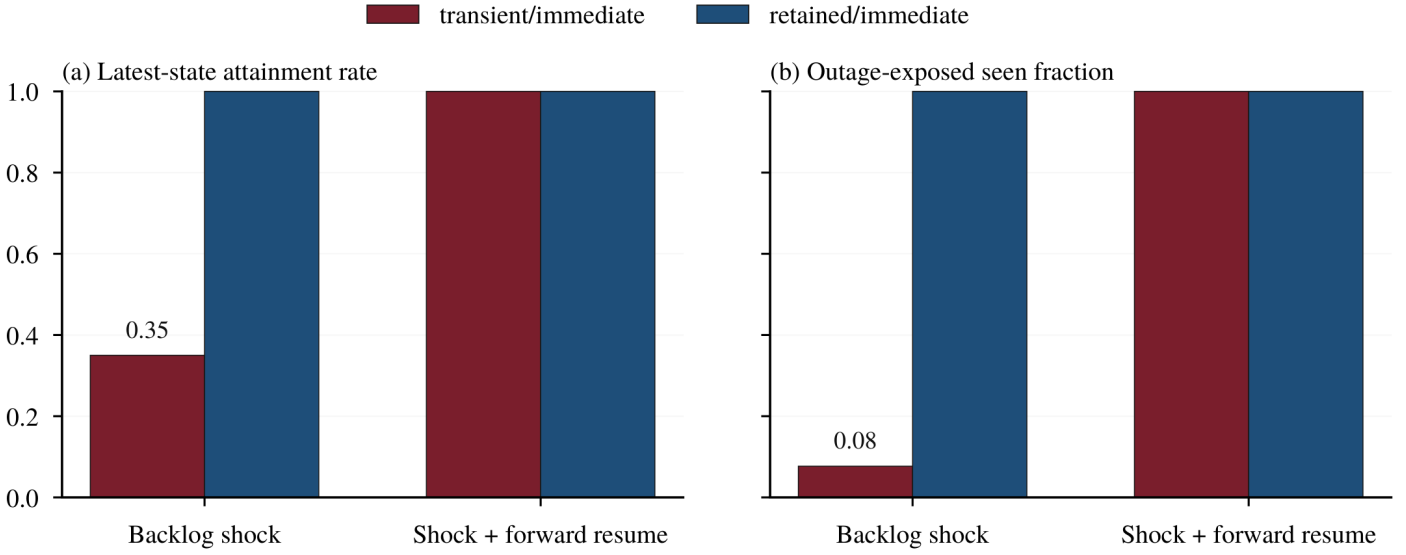


Figure 4.4: State-oriented adequacy under backlog shock and forward resume. The figure compares latest-state attainment and outage-exposed seen fraction for transient/immediate and retained/immediate handling.

The figure shows why replay relevance is conditional on the state boundary. Under backlog shock, transient handling loses state-relevant updates, while retained handling preserves them for replay. Under forward resume, later updates overtake the missed information, and both models reach full latest-state attainment. The same missed-update pattern therefore has different meaning depending on the state the process is required to preserve. Replay matters when it restores state that still counts. It matters less when newer state has already made the missed update obsolete.

Taken together, the state-oriented results suggest a restrained design implication: replay is useful when it restores state that still matters. In the continuous-update setting examined here, later updates can overtake earlier misses, while infrequent updates or long-lived state gaps may still justify retained replay. The handling model should therefore be judged by whether replay contributes to the state condition, not by event deficit or replay completion alone.

4.3 Required-effect results

Required-effect work asks whether one downstream action happens reliably and without harmful duplication. The first distinction is where the failure occurs. A downstream handling gap can remain recoverable if the event has entered the flow, while source omission leaves replay and retained handling with nothing to act on. Duplicate pressure raises a separate question: whether repeated triggers lead to harmful repeated effects, and what cost is introduced when the handling model contains them [7, 14].

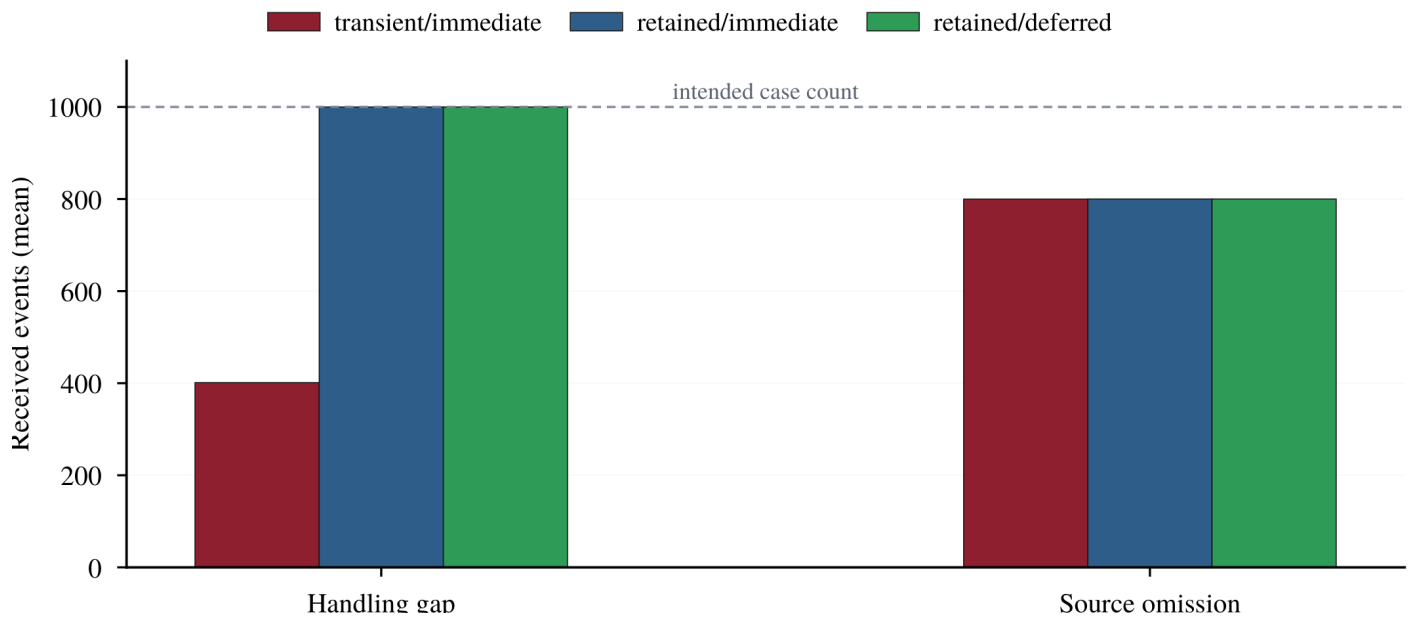


Figure 4.5: Source omission boundary measured as received events. The figure compares handling gap and source omission across the three process-handling models, with the intended case count shown as a reference.

Figure 4.5 marks the boundary of downstream recovery. Its value is that it shows where replay has an object to act on and where it does not. In a handling gap, the event has entered the flow and retained handling can recover what transient handling misses. In source omission, some cases are never emitted as events, so the failure lies before replay can help. The figure therefore separates downstream recoverability from producer-side integrity, which is essential for required-effect work.

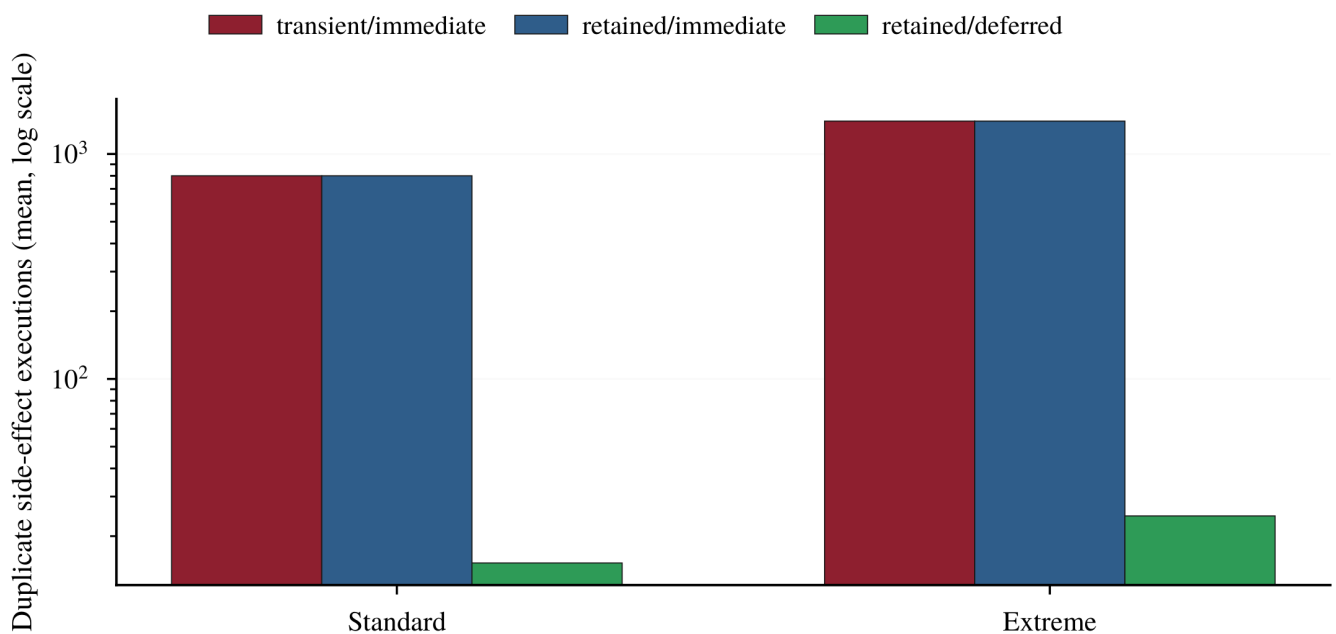


Figure 4.6: Duplicate-pressure containment. The figure shows duplicate downstream side-effects under duplicate pressure for immediate and deferred handling.

Table 4.1: Within-suite duplicate-containment trade-off under deferred settlement, shown as rounded means across ten repetitions.

Severity	Immediate dup. effects	Deferred dup. effects	Reconciliation passes	Correction rewrites	Settlement delay (s)
Standard	800	15	33	785	0.10
Extreme	1400	25	37	1375	0.06

The duplicate-pressure figure addresses another part of the required-effect boundary: immediate repeated execution versus deferred settlement. In the retained/deferred model used in the suite, repeated triggers are absorbed before final settlement, sharply reducing duplicate downstream execution. Table 4.1 shows the companion cost. Across the frozen ten-repetition batch, duplicate side-effects fall substantially under both pressure levels, while reconciliation work, correction rewrites, and settlement delay increase. Deferred handling therefore changes the burden rather than removing it, moving it out of immediate repeated execution and into later settlement work.

Taken together, the required-effect results separate concerns that are easy to blur. Replay is relevant only after a valid source event exists, while duplicate containment raises a separate settlement trade-off. In the duplicate-pressure scenario examined here, deferred handling reduces repeated downstream effects by moving work into reconciliation, correction, and settlement delay. A handling model for required-effect work should therefore be judged by both the location of the omission risk and the cost of postponing settlement.

The discussion begins from the thesis’s main contribution. The related-work chapter established the distributed-systems context most relevant to the argument. The task here is to interpret what the empirical sections add within that context. The central contribution lies in the framework developed for relating correctness boundaries to process-handling choice. The empirical results support that argument by making selected distinctions visible within the controlled research setting. Taken together, the thesis provides a clearer basis for assessing process-handling models in event-driven processes, and the discussion answers the three research questions in that order.

5.1 Answer to RQ1: which conditions define correctness across different event-driven processes?

Where a process has a deadline-constrained condition, correctness is defined by whether affected cases can still reach a valid outcome before the window closes. Delay matters because it can change the status of the outcome itself. A case may eventually complete and still fail if completion arrives after the point at which the result no longer counts. Time is therefore part of correctness, not merely a performance concern.

Where a process has a state-oriented condition, correctness is defined by whether missed or delayed updates still matter to the state the process is required to preserve or expose. A missed intermediate update is not necessarily a lasting failure, since later state may legitimately overtake it. Whether that matters depends on the specific state boundary: some boundaries are satisfied by the latest valid state, while others are affected by stale or misleading state during the disturbance. The central question is therefore whether the state condition still holds, not whether every update has been observed or replayed.

Where a process has a required-effect condition, correctness is defined by whether one downstream action happens reliably and without harmful duplication. The first distinction is where the failure occurs. Once an event has entered the flow, replay may recover a downstream handling gap. Before emission, there is no event for replay to recover. Repeated triggers create a separate risk: the action may happen more than once, so duplicate control and settlement cost become part of judging whether the condition is protected.

Taken together, these distinctions give correctness a process-level form. A process

is not correct merely because an event was delivered, retained, replayed, or eventually settled. It is correct when the condition or set of conditions it protects still holds: the outcome arrived in time, the relevant state remained acceptable, or the required downstream action happened without being lost or harmfully repeated. This is the foundation for judging the handling models discussed in the next section.

5.2 Answer to RQ2: how do process-handling models shape preservation, replay, repayment, and settlement under disturbance?

Process-handling models matter because they change which losses remain recoverable, which delays remain acceptable, and which costs are moved into later settlement. In the results presented here, three patterns stand out.

For a deadline-constrained condition, the important question is not peak pressure alone, but whether unsettled work can still be repaid before expiry. The retained/deferred backlog-shock case makes this visible: the most dramatic runtime burst does not produce the greatest final deadline harm, because endpoint damage depends on whether accumulated work can still settle in time.

For a state-oriented condition, replay value depends on whether a missed update still matters to the state condition being protected. In the continuous-update setting examined here, later updates can overtake earlier missed state, making transient handling sufficient in this setting. In other state-oriented settings, where missed updates remain relevant for longer, retained replay may still be the more appropriate model.

For a required-effect condition, downstream handling gaps and source omission are not the same kind of failure. Once an event has entered the flow, retained replay can recover missed downstream handling. If the event was never emitted, replay has no object to act on. Deferred handling adds a different trade-off: it can reduce immediate downstream pressure, but it does so by moving part of the burden into later settlement.

These are modest distinctions, but they sharpen the question of handling choice. They show that the same handling model can preserve what matters in one kind of work, arrive too late in another, or address the wrong failure altogether.

A further lesson is that recovery signs have to be read through the condition they are supposed to satisfy. A cleared backlog, a replayed event, a converged final state, or a lower duplicate count may all look favourable. None is decisive on its own. The relevant question is narrower: did the work still arrive in time, did the recovered state still matter, and did duplicate containment leave an acceptable settlement burden?

This is why the empirical suite follows both endpoint outcomes and the route by which those outcomes were reached. Endpoint results show where the process ended. Runtime and settlement traces show whether that endpoint still satisfied the condition at stake, or whether correctness had already been lost, superseded, or shifted into later work. The distinction is small in wording but important in design: a system can appear to recover while still failing the condition the process was meant to protect.

5.3 Answer to RQ3: how can explicit correctness boundaries guide process-handling choices in event-driven design?

The design implication follows from that comparison. A handling choice becomes easier to judge when it is considered at the level of a specific event-driven process, against what that process must protect. The guide developed below gives that judgment a more concrete form: state the protected condition or conditions, identify how they could fail, and choose a proportionate handling model with its trade-off made clear.

5.3.1 Boundary-first design guide

The boundary-first guide is applied to one event-driven process at a time. A larger application may contain several such processes, and a single process may carry more than one protected condition. The guide therefore asks what must remain true for the process to count as acceptable, and whether the handling choice protects that condition or set of conditions.

The compact form of the guide is shown in Table 5.1. Its purpose is to make the handling choice explicit and proportionate: the selected model should be chosen in relation to the condition or conditions at stake, rather than by treating replay, retention, or deferred settlement as inherently stronger in the abstract.

Table 5.1: Compact use of the boundary-first guide.

Step	Guiding question	Design output
Scope and state the boundary	Which event-driven process is being judged, and what must hold for its cases to count as acceptable?	A clearly scoped process and the condition or set of conditions it must protect: timely completion, acceptable state, reliable downstream effect, or more than one of these.
Identify how the boundary could fail	What kind of delay, loss, stale state, repeated action, late settlement, or missing source event would make the stated condition false?	Explicit boundary-failure modes, including whether they lie inside downstream handling or before the event enters the flow.
Choose the handling model and evidence	Which handling model is proportionate to the stated condition or conditions, and what residual risk remains?	A justified choice of transient/immediate, retained/immediate, or retained/deferred, together with any necessary supporting safeguard, the trade-off, and the metric that would reveal failure.

The full guide is provided in Appendix A, Table A.1. It expands the three steps

into the specific checks used throughout the thesis, including deadline sensitivity, state relevance, downstream recovery, duplicate harm, settlement tolerance, and source omission. Supporting safeguards are not additional handling models. They are controls that may be needed for the stated condition or set of conditions to be protected.

Source omission is included as a check rather than as a process-handling model. When the event has not been emitted, the problem lies before downstream handling. Replay, retention, and deferred settlement cannot recover work that never entered the event flow. Producer-side integrity is therefore treated separately in the following subsection.

5.3.2 Producer omission and producer-integrity controls

Downstream recovery depends on the event having entered the flow. Replay and retained handling can recover some forms of downstream loss, but they cannot recover an event that was never emitted. Where omission would materially affect the outcome, producer-side integrity controls should therefore be considered before downstream recovery is treated as sufficient.

A transactional outbox is one example. It ties event publication to the committed change that gives rise to it, helping ensure that an event is not lost before it enters the flow [23]. The framework developed here becomes directly applicable from that point onward, once questions of preservation, replay, delay, duplication, and settlement arise.

5.3.3 Illustrative design examples

The following examples are compact design readings rather than case studies. Their purpose is to show how the guide can be applied one event-driven process at a time, including cases where a process carries more than one protected condition.

5.3.3.1 Food delivery service

In a food delivery service, the authoritative order record may remain in a database while events carry surrounding work such as courier visibility, restaurant acceptance, dispatch, payment, and customer communication. These flows belong to the same order, but they do not all ask the handling model to protect the same thing.

Courier visibility is mainly a state-oriented concern. Missed intermediate location or order-status updates may be acceptable if newer state quickly restores an adequate view of the courier or order. The question is not whether every update was replayed, but whether the visible state remained adequate for its purpose. If stale state remains visible long enough to mislead dispatch, customer expectations, or operational decisions, retained handling may be more justified because the missed state still matters.

Restaurant acceptance, preparation, and courier dispatch are more time-sensitive. An acceptance or dispatch instruction may be delivered and still fail to help if the order context has moved on. Preparation may start too late for the delivery promise to be kept. In these flows, eventual completion is not enough. Some actions also

need duplicate control: a dispatch instruction may be too late in one failure mode and harmful if repeated in another.

Payment capture, refund issuance, and courier or restaurant payout raise required-effect concerns. Once the relevant business fact exists, the downstream action has to happen reliably and without harmful duplication. A missed refund, duplicated capture, or repeated payout is not a stale-state issue, but a failure to carry through the required downstream effect correctly. If timing also matters, such as when an authorisation or refund window closes, that condition should be stated alongside the required effect rather than hidden inside a general claim that the payment flow needs stronger handling.

The example should not be read as one reliability problem at system level. A courier-location update, a dispatch instruction, and a refund event may all belong to the same order, but they call for different judgments. One concerns adequate visible state, another whether an action still counts in time, and another reliable downstream completion. The handling model is justified by that process-level condition, not by appearing stronger in the abstract.

5.3.3.2 E-commerce checkout and order processing

An e-commerce checkout has a similar shape. The authoritative order, payment, and stock records may remain in transactional storage, while events carry status updates, authorisation steps, stock changes, fulfilment instructions, invoices, and refunds. The same customer order can therefore give rise to different kinds of event-driven work.

Customer-facing stock views and many order-status updates are state-oriented. They should remain current enough to be useful, but missed intermediate updates may be overtaken by later state while the authoritative record still determines the underlying commitment. The question is whether the visible state remains adequate for the decision it supports. A stale stock view may be tolerable if it is quickly corrected before checkout, but more serious if it misleads customers or downstream fulfilment for long enough to matter.

Checkout and payment authorisation are more time-sensitive. Delay or loss at that point can leave the checkout incomplete, invalid, or no longer acceptable to the customer or payment provider. Completion has to occur while the checkout or authorisation context is still valid. Duplicate control may matter as well: an authorisation that arrives too late has failed in one way, while repeated authorisation may fail in another.

Other downstream actions raise required-effect concerns. Once an order has been created, payment capture, refund issuance, invoice generation, or a fulfilment instruction may have to happen reliably and without harmful duplication. The action should not be lost, repeated, or left to ambiguous later settlement. If a carrier cutoff, cancellation window, or refund window also matters, that timing condition should be made explicit rather than folded into a general claim that the order-processing flow needs stronger handling.

The example shows the value of separating the order as a business object from the event-driven processes that act around it. A stock update, a payment authorisation, and a fulfilment instruction may all belong to the same order, but they do not protect the same condition. One concerns adequate visible state, another the validity window

for a decision, and another reliable downstream completion. The handling model should follow that process-level obligation, with supporting safeguards made explicit where the process carries more than one condition.

5.3.3.3 Reading the examples together

Read together, the examples show why the guide is applied below the level of the application as a whole. A food delivery order or an e-commerce order may involve visible state, time-sensitive decisions, required downstream effects, and supporting safeguards in the same larger system. Treating that system as one reliability problem would hide the differences the handling choice has to respect.

Appendix B extends the same reading across additional domains, showing how the protected condition, possible failure, and handling choice can be separated in ordinary design reasoning.

5.3.4 Scope and realism

The guide is meant for ordinary systems in which event-driven processes carry part of the work that matters to an outcome. The authoritative business record may remain in a transactional database, and many important decisions may still be made through ordinary application logic. What matters for the guide is narrower: whether a particular event-driven process affects whether a case reaches an acceptable result.

This defines the guide's scope. It examines one event-driven process at a time: what condition or set of conditions it must protect, how those conditions could fail, and which handling model is proportionate. Applied repeatedly, the same discipline can give a clearer view of the system's event processing as a whole, while the judgment remains process-level.

This is a modest form of design guidance. It is most useful where ordinary design language already suggests different obligations: timely action, reliable completion, recoverable state, tolerable delay, or harmful repetition. The framework gives those distinctions a more explicit shape without replacing the wider architectural judgment needed in a real system.

5.4 Relation to established work

The results clarify the thesis's relation to the work reviewed in Chapter 2. End-to-end reasoning, coordination, stream processing, recovery, and event-driven practice provide the mechanisms, guarantees, and design concerns to which the thesis responds. The contribution here is to give those concerns a process-level point of contact by asking how a familiar mechanism or guarantee bears on the condition a particular event-driven process must protect.

The closest connection is to end-to-end reasoning. The results show why a mechanism cannot be judged only by local capability. Retention and replay preserve emitted work, but their value depends on what the process needs from that preservation. They help when work can still settle in time, when missed state remains relevant, or when a downstream handling gap can still be recovered. They lose that

value when the outcome is already too late, the state has been overtaken, or the event never entered the flow. This is the process-level form of the end-to-end discipline used throughout the thesis.

That process-level reading also connects the thesis to coordination avoidance and CALM. Those accounts show that the need for coordination depends on the property a system has to preserve [8, 9]. The thesis uses a less formal, design-facing version of that discipline. It asks what a given event-driven process must protect before judging whether stronger preservation, replay, coordination, or settlement is warranted.

Stream-processing and dataflow work are relevant for a related reason: they already treat correctness, latency, and cost as connected concerns [3, 13]. The thesis brings a similar concern into event-driven process design. It asks how delay, recovery, replay, and settlement affect the condition a process must protect, rather than treating them as generic improvements or costs. A later correction, replay, or settlement step may improve one aspect of reliability while leaving another concern unresolved. Its significance depends on whether it helps preserve the condition that makes the process acceptable.

The required-effect results connect especially to idempotence and transactional event publication [7, 14, 23]. Idempotence matters when repeated handling could cause the same downstream action to happen more than once. Transactional publication matters when the event must reliably enter the flow before downstream recovery can act on it. These mechanisms address different risks within required-effect work: repeated execution in one case, producer-side non-emission in another. The thesis clarifies how each mechanism bears on the condition the process must protect.

The relation is therefore complementary. As reliability mechanisms accumulate, the basic process-level question can become easier to obscure: what must this process keep true? The question is simple, but far from trivial. It gives familiar mechanisms a clearer design role by first relating them back to the condition or set of conditions they are meant to protect.

5.5 Limitations and claim boundaries

The claims made here are restrained by design. The framework relates process-handling models to explicit process-level correctness requirements. The three correctness-boundary classes used in the thesis are working cases, not a complete taxonomy. They bring deadline validity, state relevance, and required downstream effects into view, while leaving room for other correctness concerns to require additional classes or distinctions that cut across the ones used here.

The empirical work has the same restrained scope. The suite abstracts from platform heterogeneity, operational scale, and full deployment complexity in order to compare the handling models under fixed conditions. The results should therefore be read as pattern-illuminating evidence within the examined scenarios, with the emphasis on how the handling models differ rather than on transferable performance values.

The design guide should be read accordingly. It gives the framework a compact working form, but the study does not evaluate it as a practitioner method or compare it with alternative procedures. Its role here is to show how a protected condition, a

handling model, and the resulting trade-off can be brought into view together.

Nor does the thesis rank the handling models in general. Retention, replay, immediate handling, and deferred settlement take their value from the condition at stake. Deferred settlement, for example, is appropriate only where postponing settlement is preferable to allowing immediate duplicate or repeated downstream effects. The claim is narrower: explicit correctness reasoning gives a clearer basis for judging which handling model fits the process condition.

Finally, the framework remains process-level guidance. Applied across the event-driven processes in a system, it can inform broader architectural judgment, but it does not replace the need to reason about data ownership, service boundaries, operational constraints, monitoring, and recovery practice. It illuminates and disciplines one part of that judgment: the relation between what a process must protect and how its event-driven handling should be arranged.

5.6 Ethical and professional considerations

The empirical work in this thesis does not involve human participants, personal data, or production systems, so its direct research-ethical risk is limited. The more relevant issue is professional. Reliability decisions in event-driven systems shape which failures are prevented, which are tolerated, and which are shifted into later handling. In real systems, those choices may affect people and organisations who did not choose the handling model but still bear the consequences of its limits.

This matters most where the burdens of failure are unevenly distributed. A delayed refund, missed notification, duplicated charge, or stale operational state may impose costs on customers, workers, or downstream parties who did not choose the handling model. The framework cannot decide how those costs should be balanced. It can, however, make clearer what condition is being protected and where the burden of a handling choice is likely to fall.

The framework's professional relevance is narrower and more practical. By asking what a process must protect, what can disrupt or complicate that condition, and which handling model is chosen, it makes reliability reasoning more explicit. That can support responsible design by clarifying what guarantee was intended, what risk was accepted, and where recovery or settlement work was expected to occur.

The practical implication is that a handling model should not be justified only by technical convenience or available infrastructure support. It should be justified in relation to the condition or set of conditions the process must protect, and to the consequences of failing to protect them. Making that reasoning explicit does not remove the need for wider ethical and architectural judgment. It does, however, make clearer what the handling choice is meant to secure and where the burden may fall if that protection fails.

Chapter 6

Conclusions and Future Work

6.1 Conclusion

Reliability in event-driven processes becomes clearer when the condition a process must protect is stated before the handling model is judged. In such processes, work is distributed across components and carried forward through asynchronous event handling. A mechanism may therefore preserve or recover work while leaving open whether the condition that matters still holds. The thesis starts from that gap between mechanism behaviour and process acceptability, and offers a modest way to strengthen the design judgment: ask first what the process must keep true, then judge the handling choice in that light.

The boundary-first framework gives that reasoning a process-level form. Correctness boundaries state what must hold for a process to remain acceptable, while process-handling models describe how event-driven work is carried, preserved, replayed, or settled. The controlled empirical suite examines that relation in a scoped setting, where selected disturbances make the differences between handling models visible. The results support the central claim. Deadline-constrained work shows that replay and retention matter only while unsettled work can still be repaid before expiry. State-oriented work shows that replay matters when missed state remains relevant, but less when later updates have overtaken it. Required-effect work shows that downstream replay can recover handling gaps after emission, but not source omission, and that deferred settlement reduces immediate duplicate pressure by moving part of the burden into later reconciliation.

The contribution is a process-level framework for relating correctness boundaries to process-handling choice. It connects the condition or set of conditions a process must protect, what can disrupt or complicate them, the handling model chosen, and the resulting trade-off. The guide gives this reasoning a more concrete form. Its design value is modest: it helps make the basis for a handling choice clearer by asking first what must remain true for the process to count as acceptable, and then which handling model is proportionate to that requirement. Clarity about what must hold gives clarity about how it should be handled.

6.2 Placing the thesis in perspective

The starting point for the thesis was a difficulty that remains easy to miss in event-driven design. The fact that an event has been emitted, retained, replayed, or eventually handled does not by itself show that the process has remained correct. Cor-

rectness depends on whether the handling preserved the condition that made the process acceptable in the first place. The thesis contributes by making that dependency explicit and by using it to judge process-handling choices.

This gives the work a wider relevance beyond the empirical suite. Modern event-driven systems give designers many ways to preserve and recover work, but those mechanisms do not settle the design question on their own. Their value depends on how they bear on the process at hand: what can still be recovered, what has arrived too late, what has only shifted cost elsewhere, and what never entered the event flow at all.

In that sense, the thesis sits between mechanism-level reliability and process-level correctness. It does not propose a new event-processing mechanism. It gives a way to interpret existing mechanisms in relation to the work they are meant to protect. This is where the empirical results matter. They show that favourable signs of recovery are not self-explanatory. The same apparent recovery may preserve the relevant condition, arrive after that condition has already failed, or move part of the burden into later settlement.

This perspective also clarifies the thesis's limits. The boundary classes used here are working distinctions, not a complete theory of distributed correctness. Other processes may raise concerns that do not fit neatly into those classes. The point is not that every such case can be absorbed into the framework, but that the thesis gives a way to begin the judgment: by asking what condition the process must protect and what kind of handling would preserve it.

Seen in this light, the thesis offers a modest contribution to design knowledge in event-driven systems. It shows that reliability is not a matter of strength in the abstract, but of suitability: whether a handling model preserves the condition at stake, addresses the failures that threaten it, and carries an acceptable cost.

6.3 Future Work

Future work can move the argument from controlled comparison toward the settings in which event-driven handling choices are actually made. One direction is to study design practice itself. Architecture discussions, design reviews, and incident reports could show how teams move from business or system requirements to process-level judgments about what must remain true, where recovery is expected to occur, and what settlement cost is acceptable. This would test whether the reasoning made explicit in the thesis corresponds to reasoning that practitioners already perform implicitly, and where the framework would need adjustment.

Another direction is to evaluate the design guide as a practical aid. Such a study could ask whether the guide helps teams state protected conditions, compare handling models, and recognise mixed obligations within the same application. The relevant question would not be whether the guide produces one correct answer, but whether it makes the grounds of a handling choice clearer and easier to discuss.

Modern event-streaming and stream-processing platforms are a promising setting for this work. They place more recovery, replay, ordering, and state management behind platform controls [13]. Further research could examine how teams connect those controls to process-level correctness requirements, and how they decide what

belongs in platform configuration, application-side safeguards, monitoring, or operational recovery.

Future work could also extend the framework into operations. Once a process-level condition has been stated, monitoring and incident review can ask whether that condition held during disturbance, not only whether events moved through the system. This may change what evidence is collected after failure and how recovery behaviour is interpreted.

Finally, further work could examine processes whose correctness depends on several connected conditions at once. Ordering, auditability, cross-case relationships, and grouped downstream effects may require boundary statements to be combined or refined. These cases would be valuable because they test the framework at the point where a single process-level judgment may no longer be enough.

Together, these directions would place the framework in real design, platform, and operational settings. They would ask not only whether explicit correctness boundaries help clarify handling choices, but also where the framework itself needs to grow.

Appendix A. Boundary-first design guide

Table A.1: Boundary-first guide for process-handling choice.

Design check	Process-level question	What the answer indicates	Handling implication
Scope the process	Which specific event-driven process is being judged?	The guide applies to one process at a time, not to the whole application at once.	If several protected conditions appear, state them explicitly. Some may require separate process-level judgments, while others may need to be protected together by the same handling choice or by supporting safeguards.
State the boundary	What must hold for a case in this process to count as acceptable?	The relevant protected condition or conditions are made explicit: timely completion, acceptable state, reliable downstream effect, or more than one of these.	Do not choose replay, retention, immediate handling, deferred settlement, or supporting safeguards before this condition or set of conditions is clear.
Identify how the boundary could fail	What kind of failure would make the stated condition or conditions false?	The relevant failure mode may be delay, backlog, missed handling, stale state, repeated action, late settlement, or a missing source event.	Mechanisms should be judged against the boundary-failure mode they can actually address.
Check deadline sensitivity	Would completion still count if it arrived late?	Timeliness is part of correctness, not merely a performance concern.	Retention and replay help only if preserved work can still settle before expiry. Use immediate or deadline-aware handling when lateness invalidates the case.
Check state relevance	Can later state overtake a missed or delayed update?	Replay matters when the missed update remains relevant to the visible or resulting state.	Transient/immediate may be sufficient for rapidly superseded state. Retained/immediate is stronger where missed state can remain stale or harmful.
Check downstream recovery	Was the event emitted, but missed or delayed downstream?	The failure is inside the consumer-side handling path.	Retained/immediate can recover emitted work through replay when the downstream effect still has to happen.
Check duplicate harm	Would repeated execution of the downstream action be harmful?	Correctness may require not only that the effect happens, but that it does not happen repeatedly in a harmful way.	Retained/deferred may be appropriate when duplicate effects are costly and later settlement is acceptable.
Check settlement tolerance	Is delayed settlement acceptable for this process?	Deferred handling can reduce duplicate execution, but may add reconciliation work and settlement delay.	Use retained/deferred only when the cost of later settlement is preferable to the cost of harmful duplicate effects.
Check source omission	Could the event fail to enter the flow at all?	The failure lies before downstream handling. Replay and retention cannot recover what was never emitted.	Treat this as a producer-integrity concern before applying the downstream process-handling guide.
Choose the handling model	Which handling model is proportionate to the stated condition or conditions, given the failure mode being addressed?	The model should fit the condition or conditions at stake, not appear strong in the abstract.	Choose transient/immediate, retained/immediate, or retained/deferred according to what the process must protect. Record any necessary supporting safeguard, the residual risk, and the metric that would reveal failure.

Appendix B. Supplementary design examples

Table B.1: Design scenario summary

System	State-oriented	Deadline-constrained	Required-effect
Food delivery	Courier visibility	Acceptance and dispatch timing	Payment, refund, or payout handling
Online reservation	Availability view	Confirmation before hold expiry	Reminder and calendar update
E-commerce checkout	Stock and order-status view	Checkout and payment authorisation	Payment capture, refunds, invoicing, or fulfilment instructions
Subscription billing	Account and access-status view	Renewal and status-transition timing	Receipts, reminders, and billing follow-ons
Perishable refrigerated-goods logistics	Temperature and shipment-condition visibility	Intervention before spoilage threshold	Reroute, notification, and compliance action

Each row maps one illustrative example to the three correctness categories used in the thesis.

Appendix C. Empirical suite synopsis

The empirical suite is a controlled event-processing environment used to examine how the process-handling models behave under selected correctness boundaries and disturbances. Its purpose is to make a small number of handling differences observable: whether work can still settle before deadline expiry, whether missed state remains relevant, whether emitted work can be recovered through replay, whether source omission leaves downstream replay with nothing to act on, and whether duplicate containment shifts cost into later reconciliation.

Each scenario cell combines three elements: a boundary module, a disturbance profile, and a process-handling model. The boundary modules correspond to the three working boundary classes used in the thesis: deadline-constrained, state-oriented, and required-effect. The process-handling models are transient/immediate, retained/immediate, and retained/deferred, except in the state-oriented module, where the comparison focuses on transient/immediate and retained/immediate because duplicate containment and delayed settlement are not the state-oriented contrast being examined.

A run begins with a producer emitting the configured workload into the event substrate. Redis Pub/Sub is used for transient publication, while Redis Streams is used where emitted events must remain available for replay. The configured disturbance then affects the run: processing may pause or slow down, work may accumulate, emitted events may be missed downstream, source omission may prevent some intended cases from being emitted as events, or repeated triggers may create duplicate-effect pressure. The selected handling model processes, preserves, replays, or settles the work according to the scenario.

Reporting scripts collect both endpoint outcomes and runtime traces. Deadline-constrained scenarios report measures such as deadline miss rate, completion-in-time rate, unresolved burden, doomed burden, repayment pattern, and resolution pulse. State-oriented scenarios report latest-state attainment and outage-time exposure. Required-effect scenarios report attainment, received-event counts, duplicate side effects, reconciliation work, and settlement delay. Each scenario cell was repeated ten times, giving 34 cells and 340 completed runs.

Figure C.1 gives a compact overview of this run flow. Tables C.1–C.4 give the operational definitions, scenario parameters, disturbance-channel mapping, and evaluation controls used in the reported suite. The reported values should be read as controlled comparisons within this research setting, not as quantitative predictions for deployed event-driven systems.

Empirical Suite Execution Pipeline

Canonical run flow from scenario profile to aggregated outputs

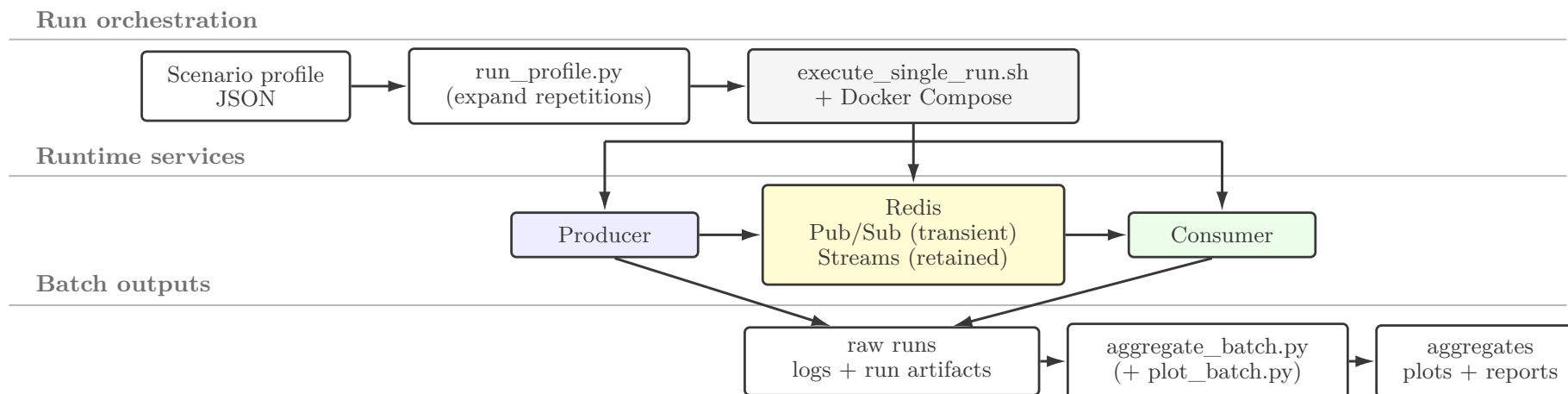


Figure C.1: Empirical suite execution pipeline from scenario profile to aggregated outputs.

Suite run stability. Across repeated runs, the suite shows a stable central pattern, with variation concentrated in a limited number of timing-sensitive edge cases. In most cells, mean and median remain close, indicating that the typical run and the aggregate tendency align well. Where they separate more clearly, the affected cells are usually those closest to a fixed deadline boundary or most exposed to interruption and replay timing. In those cases, small shifts in scheduling or recovery timing can move cases from just before to just after the boundary, increasing spread without altering the broader comparative pattern. The resulting edge variance does not unsettle the main patterns of the suite, which remain stable and clear across repeated full runs. The thesis itself rests on the frozen canonical results linked from the empirical suite page, while new reruns of the suite can be executed and compared against those frozen aggregates to confirm that the same comparative patterns continue to hold.

Table C.1: Operational definitions of reported metrics.

Metric	Computation rule	Used for	Interpretation
Deadline miss rate	Expired cases divided by produced cases.	Deadline-constrained	Share of cases that expired before deadline-valid completion.
Completion-in-time rate	Cases completed before expiry divided by produced cases.	Deadline-constrained	Share of cases that reached a valid outcome in time.
Unresolved burden	Produced cases not yet attained at a runtime sample.	Deadline-constrained	Work still unsettled during the run.
Doomed burden	Unsettled produced cases whose deadline has already passed at a runtime sample.	Deadline-constrained	Work that remains unresolved after timely correctness is no longer possible.
Repayment pattern	Reduction of unresolved burden before cases cross the deadline, read with completion-in-time and doomed burden.	Deadline-constrained	Shows whether accumulated work can still be worked down in time.
Resolution pulse	Completions and expiries counted per fixed runtime bin.	Deadline-constrained	Burstiness of settlement activity during recovery.
Required-effect attainment	Cases whose required downstream effect occurred divided by produced cases.	Required-effect	Share of required effects that happened.
Received-event count	Events observed by the consumer after omission or duplicate pressure.	Required-effect	Separates downstream loss from source omission and duplicate triggering.
Duplicate side-effect count	Repeated downstream executions beyond the intended effect.	Required-effect	Measures harmful duplicate execution.
Correction rewrites and reconciliation passes	Deferred overwrites and settlement sweeps recorded by the retained/deferred model.	Required-effect	Measures the cost introduced by duplicate containment.
Settlement delay	Time from producer completion to last recorded attainment.	Required-effect	Shows whether deferred handling shifts cost into later settlement.
Latest-state attainment rate	Cases whose resulting latest state is acceptable divided by produced cases.	State-oriented	Measures final state adequacy.
Outage-exposed seen fraction	Seen outage-relevant updates divided by expected outage-relevant updates.	State-oriented	Separates final adequacy from outage-time exposure.

Table C.2: Scenario and parameter specification.

Module	Scenario family	Cells	Workload	Boundary parameter	Disturbance specification	Handling models	
Deadline-constrained	Baseline	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 30 s observation window.	No disturbance.	T/I, R/D.	R/I,
Deadline-constrained	Backlog shock	3	1000 cases, 5 ms base emission interval.	1.2 s deadline window, 35 s observation window.	2.8 s interruption triggered at 20% progress, with overload emission interval of 1 ms for 6 s starting at 1.5 s.	T/I, R/D.	R/I,
Deadline-constrained	Moderate degradation	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 35 s observation window.	Sustained 8 ms processing delay over 8.0 s.	T/I, R/D.	R/I,
Deadline-constrained	High degradation	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 35 s observation window.	Sustained 50 ms processing delay over 8.0 s.	T/I, R/D.	R/I,
Required-effect	Baseline	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	No disturbance.	T/I, R/D.	R/I,
Required-effect	Handling gap, standard	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	3.5 s handling interruption starting at 2.0 s.	T/I, R/D.	R/I,
Required-effect	Handling gap, extreme	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	5.0 s handling interruption starting at 1.5 s.	T/I, R/D.	R/I,
Required-effect	Source omission	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	20% of intended cases omitted before emission, leaving 800 emitted cases.	T/I, R/D.	R/I,
Required-effect	Duplicate pressure, standard	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	40% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.	T/I, R/D.	R/I,
Required-effect	Duplicate pressure, extreme	3	1000 cases, 5 ms emission interval.	Required effect, no deadline window.	70% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.	T/I, R/D.	R/I,
State-oriented	Backlog shock	2	300 cases $\times$ 3 updates (900 emitted updates), 0.3 obsolete replay fraction.	Latest-state adequacy after final update.	2.4 s interruption triggered at 80% progress.	T/I, R/I.	
State-oriented	Backlog shock with forward resume	2	300 cases $\times$ 4 updates (1200 emitted updates), 0.3 obsolete replay fraction.	Latest-state adequacy after final update.	0.8 s interruption triggered at 60% progress, followed by forward resumption.	T/I, R/I.	

Table C.3: Disturbance-to-channel map.

Scenario family	Primary disturbance locus	Observable effect	Replay expectation
Handling gap	Downstream handling interruption.	Events are emitted, but missed or delayed by the consumer.	High for retained modes when the event remains available.
Source omission	Producer-side non-emission.	Some intended cases are never emitted as events.	None. Replay cannot recover an event that was never emitted.
Backlog shock	Temporary processing interruption and catch-up pressure.	Work accumulates and then resolves in bursts.	Depends on whether the remaining validity or relevance window is still open.
Degradation	Sustained processing slowdown.	Work progresses under continuous latency pressure.	Limited by deadline windows and accumulated doomed burden.
Duplicate pressure	Repeated triggering of the same obligation.	Duplicate downstream execution risk increases.	Deferred handling can absorb repeated triggers before settlement.
State backlog shock	Outage over a state-relevant update window.	State may be unseen, lost, or restored by later updates.	High when the missed state remains relevant to correctness.

Table C.4: Evaluation controls and scope protection.

Control objective	Operational check	Evidence in the suite	Residual limitation
Pattern stability	Each scenario cell was repeated ten times.	34 cells and 340 completed runs.	Single-suite setting, not an external validity claim.
Channel separation	Source omission and handling gaps are separate scenario families.	Required-effect module.	Does not model every producer or broker failure mode.
Boundary discipline	Deadline, state, and required-effect modules are reported separately.	Module structure and metric groups.	Larger applications may contain several process-level obligations.
Traceability	Raw runs, run specifications, metric summaries, aggregate outputs, and plots are preserved in the project artifact.	Batch folder structure.	Reproduction still depends on the local Docker/Redis runtime.
Interpretive restraint	Results are read as controlled comparisons.	Method, Results, and Limitations wording.	No claim of platform benchmarking or universal quantitative law.

Supplementary material

Two supplementary resources accompany the thesis. The companion site gives a brief introduction to the thesis, provides access to the thesis PDF, links to selected resources for further reading on event-driven systems, and also links to the empirical suite repository. The empirical suite page provides access to the suite used in the thesis, including the frozen canonical results and material for rerunning and comparing the suite.

- Companion site: <https://kh3rm.github.io/thesis-companion-site/>
- Empirical suite page: <https://github.com/kh3rm/thesis-empirical-suite>

The thesis text remains the authoritative account of the argument, method, results, and interpretation.

References

- [1] Laigner, R., Almeida, A. C., Assuncao, W. K. G., and Zhou, Y. An empirical study on challenges of event management in microservice architectures. *ACM Transactions on Software Engineering and Methodology*, advance online publication, 2025. DOI: 10.1145/3776581.
- [2] Laigner, R., Zhou, Y., Salles, M. A. V., Liu, Y., and Kalinowski, M. Data management in microservices: State of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, 14(13), 3348–3361, 2021. DOI: 10.14778/3484224.3484232.
- [3] Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernandez-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., and Whittle, S. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803, 2015. DOI: 10.14778/2824032.2824076.
- [4] Alpern, B., and Schneider, F. B. Defining liveness. *Information Processing Letters*, 21(4), 181–185, 1985. DOI: 10.1016/0020-0190(85)90056-0.
- [5] Saltzer, J. H., Reed, D. P., and Clark, D. D. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4), 277–288, 1984. DOI: 10.1145/357401.357402.
- [6] Saito, Y., and Shapiro, M. Optimistic replication. *ACM Computing Surveys*, 37(1), 42–81, 2005. DOI: 10.1145/1057977.1057980.
- [7] Garcia-Molina, H., and Salem, K. Sagas. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 249–259, 1987. DOI: 10.1145/38713.38742.
- [8] Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., and Stoica, I. Coordination avoidance in database systems. *Proceedings of the VLDB Endowment*, 8(3), 185–196, 2014. DOI: 10.14778/2735508.2735509.
- [9] Hellerstein, J. M., and Alvaro, P. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9), 72–81, 2020. DOI: 10.1145/3369736.
- [10] Eugster, P. T., Felber, P. A., Guerraoui, R., and Kermarrec, A.-M. The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2), 114–131, 2003. DOI: 10.1145/857076.857078.
- [11] Patterson, D., Brown, A., Broadwell, P., Candea, G., Chen, M., Cutler, J., Enriquez, P., Fox, A., Kiciman, E., Merzbacher, M., Oppenheimer, D., Sastry, N., Tetzlaff, W., Traupman, J., and Treuhaft, N. Recovery Oriented Computing

- (ROC): Motivation, definition, techniques, and case studies. UCB/CSD-02-1175, University of California, Berkeley, 2002.
- [12] Dean, J., and Barroso, L. A. The tail at scale. *Communications of the ACM*, 56(2), 74–80, 2013. DOI: 10.1145/2408776.2408794.
- [13] Fragkoulis, M., Carbone, P., Kalavri, V., and Katsifodimos, A. A survey on the evolution of stream processing systems. *The VLDB Journal*, 33(2), 507–541, 2024. DOI: 10.1007/s00778-023-00819-8.
- [14] Helland, P. Idempotence is not a medical condition. *ACM Queue*, 10(4), 30–46, 2012. DOI: 10.1145/2181796.2187821.
- [15] Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., and Rabiser, R. A comprehensive benchmarking analysis of fault recovery in stream processing frameworks. *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems*, 171–182, 2024. DOI: 10.1145/3629104.3666040.
- [16] Tahir, J., Mayer, R., Doblander, C., and Jacobsen, H.-A. How reliable are streams? End-to-end processing-guarantee validation and performance benchmarking of stream processing systems. *Proceedings of the VLDB Endowment*, 18(3), 585–598, 2025. DOI: 10.14778/3712221.3712227.
- [17] Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77, 2007. DOI: 10.2753/MIS0742-1222240302.
- [18] Hevner, A. R., March, S. T., Park, J., and Ram, S. Design science in information systems research. *MIS Quarterly*, 28(1), 75–105, 2004. DOI: 10.2307/25148625.
- [19] Docker documentation. Docker Compose. Available at: <https://docs.docker.com/compose/>.
- [20] Redis documentation. Redis Pub/Sub. Available at: <https://redis.io/docs/latest/develop/pubsub/>.
- [21] Redis documentation. Redis Streams. Available at: <https://redis.io/docs/latest/develop/data-types/streams/>.
- [22] Shapiro, M., Pregelica, N., Baquero, C., and Zawirski, M. Conflict-free replicated data types. *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems*, 386–400, 2011. DOI: 10.1007/978-3-642-24550-3_29.
- [23] AWS Prescriptive Guidance. Transactional outbox pattern. Amazon Web Services, updated 2024. Available at: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>.